Represent tokens in context

Tokenisation and Positional Representation

From discrete symbols to contextual representations

Jonathon Hare

Vision, Learning and Control
University of Southampton

Models operate on representations, not raw text

raw text $\longrightarrow$ tokens $\longrightarrow$ token IDs $\longrightarrow$ vectors $\longrightarrow$ model

Tokenisation

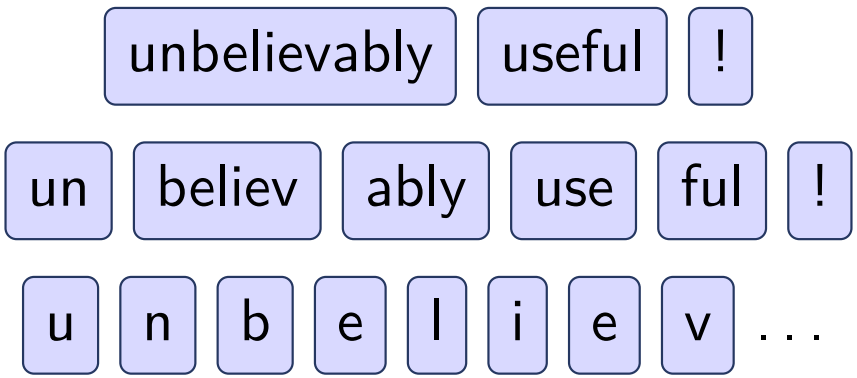
A deterministic mapping between a string and a sequence of symbols drawn from a finite vocabulary.

The mapping determines what the model can see, how long its inputs are, and which symbols share parameters.

Tokenisation is a modelling decision

The same string admits many reasonable segmentations:

`unbelievably useful!`



There is no universally correct unit. The useful choice depends on the data, the task, and the computational budget.

Words are an unstable unit of representation

What counts as one word?

- `don't`, `state-of-the-art`, `Jon's`
- `AICE3002/6001`, `3.14`, `hello@example.org`
- punctuation, emoji, URLs, code, and hashtags
- languages that do not mark every word boundary with a space
- productive morphology: `train`, `trained`, `retraining`, `trainability`

A rule such as "split on spaces" is simple, but it does not solve the representation problem.

Normalisation is not neutral

Before tokenisation, a pipeline may change the string:

- Unicode normalisation;
- case folding;
- standardising whitespace or punctuation; and
- replacing URLs, usernames, or numbers with special symbols.

Each choice trades sparsity against information:

$$\texttt{US} \neq \texttt{us} \qquad \texttt{Polish} \neq \texttt{polish} \qquad \texttt{1.0} \neq \texttt{1000}$$

The normaliser and tokenizer are therefore part of the model specification, not incidental data cleaning.

A vocabulary maps tokens to integer IDs

Let $\mathcal{V}$ be a finite vocabulary. The tokenizer produces

$$(t_1, \ldots, t_n) \quad \longrightarrow \quad (i_1, \ldots, i_n), \qquad i_j \in \{0, \ldots, |\mathcal{V}| - 1\}.$$

token	`[PAD]`	`[UNK]`	`deep`	`learn`	`ing`
ID	0	1	284	931	47

`deep learning` $\longrightarrow$ deep | learn | ing $\longrightarrow (284, 931, 47)$

IDs are arbitrary labels. ID 931 is not "larger" or more similar than ID 284.

Special tokens communicate structure

A vocabulary normally reserves symbols that do not correspond directly to a span of ordinary text:

- `[UNK]`: a symbol the vocabulary cannot represent;
- `[PAD]`: fills unused positions in a batch;
- `[SOS]` / `[EOS]`: sequence boundaries;
- `[SEP]`: separates segments; and
- `[MASK]`: hides a token for a pretraining objective.

Their meaning comes from how the model is trained and how the loss and masks treat them.

Batches require padding and a length signal

	1	2	3	4	5
input A	`deep`	`learn`	`ing`	`[PAD]`	`[PAD]`
mask A	1	1	1	0	0
input B	`sub`	`word`	`token`	`is`	`ation`
mask B	1	1	1	1	1

Padding makes tensors rectangular. A mask says which positions contain data.

Do not infer length from the padding vector

A special embedding for `[PAD]` does not, by itself, stop later computations from using padded positions.

A word vocabulary has an open-vocabulary problem

Even a large training corpus will not contain every future word:

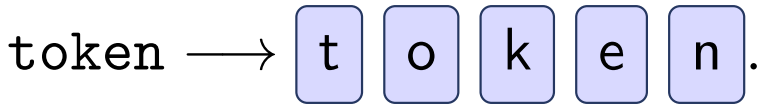
- names and places;
- new products, slang, and technical terms;
- spelling variants and typing errors; and
- unseen inflections or compounds.

Mapping all of them to `[UNK]` discards their internal structure:

$$\texttt{retokenisation}, \texttt{tokenizer}, \texttt{tokenised} \longrightarrow \texttt{[UNK]}, \texttt{[UNK]}, \texttt{[UNK]}.$$

Characters provide coverage at the cost of length

A character vocabulary is small and can compose unseen words:



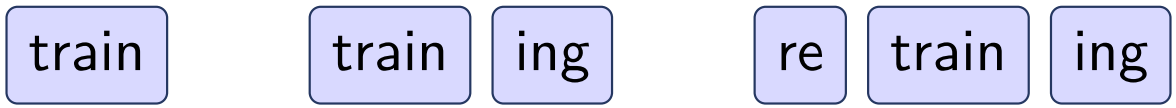
But:

- sequences become much longer;
- the model must learn common fragments repeatedly; and
- a character is not always one byte or one visible glyph.

Word tokens reuse whole meanings efficiently; character tokens provide robust coverage. Subword tokenisation aims for a compromise.

Subwords balance coverage and sequence length

Frequent strings can be single tokens; rarer strings are composed from smaller pieces:



A subword vocabulary:

- shares parameters across related surface forms;
- reduces or removes unknown tokens; and
- keeps common expressions shorter than character tokenisation.

The pieces need not be linguistically meaningful morphemes: they are learned to represent the training corpus efficiently.

Byte-pair encoding learns pieces by merging

Start with a base vocabulary of small symbols, then repeatedly:

1. count adjacent symbol pairs in the training corpus;
2. choose the most frequent pair (using a fixed tie-break rule);
3. replace every occurrence of that pair with a new symbol; and
4. add the merge to an ordered list of rules.

Stop after reaching a chosen vocabulary size or number of merges.

Adapted from Sennrich, Haddow and Birch, "Neural Machine Translation of Rare Words with Subword Units", 2016.

BPE begins with the smallest available symbols

Consider the weighted training vocabulary

$$\texttt{play:5}, \quad \texttt{playing:2}, \quad \texttt{played:2}, \quad \texttt{player:2}.$$

Initially each word is split into characters (plus a word-boundary marker):

$$\texttt{play} \rightarrow \texttt{p l a y </w>}$$
$$\texttt{playing} \rightarrow \texttt{p l a y i n g </w>}.$$

The pairs `p l`, `l a`, and `a y` each occur 11 times. Suppose a deterministic left-to-right tie-break chooses `p l`.

A merge creates one new vocabulary item

The first rule replaces `p l` by `pl`:

$$\texttt{p l a y} \longrightarrow \texttt{pl a y}.$$

Recount pairs, then continue:

step	merge rule	representation of `playing`
0	—	`p l a y i n g`
1	`p + l` → `pl`	`pl a y i n g`
2	`pl + a` → `pla`	`pla y i n g`
3	`pla + y` → `play`	`play i n g`

The base symbols remain available, so uncommon strings can still be composed.

Encoding uses the learned rules, not new corpus counts

Once training is complete, freeze the vocabulary and ordered merge rules. For an unseen string `plays`:

$$\texttt{p l a y s} \longrightarrow \texttt{pl a y s} \longrightarrow \texttt{pla y s} \longrightarrow \boxed{\text{play}}\ \boxed{\text{s}}.$$

We do *not* choose new merges from the evaluation example. Training and inference must use exactly the same tokenizer.

The merge budget controls granularity

More merges give shorter sequences and a larger embedding table; fewer merges give longer sequences and more sharing through small pieces.

BPE is one subword construction, not the definition

BPE learns an ordered sequence of frequent pair merges.

WordPiece learns a subword vocabulary and commonly encodes using greedy longest matching; BERT uses this family.

Unigram LM assigns probabilities to candidate pieces and chooses a likely segmentation; training prunes pieces from a larger vocabulary.

Different algorithms can produce different, equally valid segmentations. Always report the actual tokenizer and vocabulary, rather than saying only "we used subwords".

Bytes provide a universal base alphabet

UTF-8 represents text as bytes. Starting from the 256 possible byte values means every input string can be represented without an unknown character.

- Byte-level BPE merges frequent byte sequences into larger tokens.
- A visible character may occupy several bytes.
- A learned token may include punctuation or a leading space.
- SentencePiece instead exposes whitespace as a symbol and can train BPE or unigram models directly from raw text.

Token $\neq$ word $\neq$ character $\neq$ byte

Token counts should not be interpreted as word counts unless the tokenizer actually uses words.

Vocabulary size and sequence length trade off

unit	vocabulary	sequence length	unseen strings
word	large	short	problematic
character	small	long	usually covered
subword	medium	medium	composed
byte	256 base symbols	potentially longest	always covered

A larger vocabulary increases the parameters in the embedding/output tables. A longer sequence increases the work done by the sequence model. Later, self-attention over n positions will require an $n \times n$ matrix: tokenisation can therefore change computational cost substantially.

Fragmentation is not distributed evenly

A vocabulary reflects the frequencies in the corpus used to learn it.

- Common languages and domains receive longer, reusable pieces.
- Rare languages, names, and specialist terms may fragment more heavily.
- Under a fixed token limit, heavier fragmentation means less source text fits into the model.
- Changing the tokenizer changes the inputs, compute, and often model quality—even when the architecture is unchanged.

Inspect token lengths on the populations and domains that matter for the task; an average over the training corpus can hide important differences.

The tokenizer is fitted, frozen, and versioned

Vocabulary learning uses data and must respect the experimental design:

- learn or select the tokenizer using training data (or a declared external pretraining corpus);
- do not adapt merge rules or vocabulary to the test set;
- fix normalisation, special-token conventions, and maximum length; and
- save the tokenizer with the model checkpoint.

For a pretrained model, its embedding table expects the matching tokenizer. Replacing the tokenizer usually changes the meaning and number of its input IDs.

Token IDs become learned vectors

As in the embeddings lecture, an embedding table $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ maps each ID to a vector:

$$i_j \longmapsto \mathbf{e}_j = E[i_j] \in \mathbb{R}^d.$$

For a batch of ID sequences with shape $B \times n$, an embedding lookup returns a tensor with shape

$$B \times n \times d.$$

The lookup is differentiable with respect to the selected rows of E; the integer IDs themselves are not learned or differentiated.

Token embeddings alone do not encode order

A lookup returns one vector per token, but the vector for `dog` is the same wherever it occurs:

`dog bites person` versus `person bites dog`

If a model treats these vectors as an unordered set, permuting the tokens merely permutes the outputs.
Recurrence encodes order through sequential processing; convolutions use ordered local neighbourhoods.

No explicit positional encoding (NoPE)

Some causal decoder-only Transformers omit position embeddings. Their mask exposes a different number of predecessors at each position, allowing the model to learn implicit position information.

A fully bidirectional self-attention layer with no position-dependent signal remains permutation equivariant.

Haviv et al., "Transformer Language Models without Positional Encodings Still Learn Positional Information", 2022.

Learned absolute position embeddings

Give position j a learned vector $\mathbf{p}_j \in \mathbb{R}^d$ and combine it with the token embedding:

$$\mathbf{x}_j = \mathbf{e}_j + \mathbf{p}_j.$$

The original BERT encoder uses this scheme: a second lookup table contains one vector for each supported absolute position. Addition preserves shape, so the sequence remains $n \times d$.

- The model can learn position-specific behaviour from data.
- The table introduces a configured maximum sequence length.
- Positions not trained sufficiently may have poorly estimated vectors.

Token and position vectors must share the same dimensionality to be added.

Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", 2019.

Sinusoidal positions use fixed waves at many scales

For model width d, position j, and coordinate-pair index k:

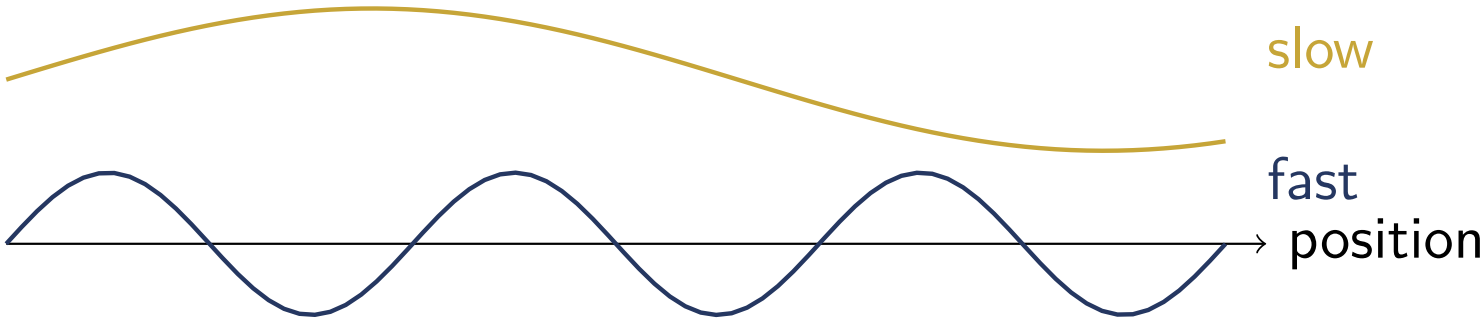
$$\mathrm{PE}(j, 2k) = \sin\left(j/10000^{2k/d}\right),$$
$$\mathrm{PE}(j, 2k+1) = \cos\left(j/10000^{2k/d}\right).$$

- j is the integer position in the sequence.
- $k = 0, \ldots, d/2 - 1$ selects a sine/cosine pair.
- Different pairs vary at different rates across the sequence.
- The vectors are generated, not learned from a position table.

Vaswani et al., "Attention Is All You Need", 2017.

Why use several positional frequencies?

A single slowly varying coordinate cannot uniquely describe many positions; a single rapidly varying coordinate repeats too often.



Combining phases at multiple scales gives each position a distinctive pattern. The construction can generate vectors beyond the training length, although the rest of the model is not guaranteed to generalise there.

Relative positions describe offsets, not addresses

For many sequence operations, the useful fact is that token i is two places before token j, rather than their absolute addresses.

- Relative schemes use a representation of the offset $i - j$.
- This can be added as a bias or incorporated into the vectors being compared.
- Rotary position embeddings (RoPE) rotate vector pairs by a position-dependent angle, making their comparison depend on relative offset.

We will return to where such information enters when we study attention. The key distinction is *absolute position* j versus *relative position* $i - j$.

Padding, positions, and masks answer different questions

Token ID Which vocabulary item is present?

Position ID Where does this item occur in the sequence?

Padding mask Does this tensor location contain a real input token?

token	[SOS]	deep	learn	ing	[PAD]
position	0	1	2	3	4
mask	1	1	1	1	0

Implementations differ in whether padded positions receive ordinary position IDs, but masking must still ensure that padding does not act like content.

Static embeddings cannot resolve context

A lookup table assigns one vector to each token ID:

`the river` <u>`bank`</u> `the investment` <u>`bank`</u>

Both occurrences initially retrieve the same vector $E[i_{\texttt{bank}}]$.
A *contextual* representation is produced after a sequence model combines information across positions:

$$\mathbf{h}_j = f_j(\mathbf{x}_1, \ldots, \mathbf{x}_n).$$

The two occurrences can therefore have different $\mathbf{h}_j$, even though they began with the same token embedding.

Tokenisation does not itself create meaning in context

$$\text{text} \xrightarrow{\text{tokenizer}} \text{IDs} \xrightarrow{\text{lookup + position}} \mathbf{x}_{1:n} \xrightarrow{\text{sequence encoder}} \mathbf{h}_{1:n}$$

- The tokenizer is a discrete, normally fixed preprocessing model.
- The embedding table learns a static vector for each vocabulary item.
- The encoder learns to make each vector depend on surrounding tokens.
- Subword pieces can be contextualised and then combined for a word- or sequence-level task.

RNNs, which we study next, and Transformer encoders, which come later, provide different mechanisms for constructing $\mathbf{h}_{1:n}$.

The representation pipeline

1. Normalise and tokenise text using a fixed, documented model.
2. Map tokens to IDs, adding boundary symbols, padding, and masks.
3. Look up token vectors and make order observable.
4. Use a sequence encoder to turn static inputs into contextual states.

The central trade-off

Larger reusable pieces shorten sequences; smaller pieces improve coverage and parameter sharing. The tokenizer decides where that compromise is made.